

stack v1.0.0 — Full Documentation

One skill to install them all.

`/stack` installs Longhand, Context-Mode, and Hardgate as a coordinated Claude Code stack — with timestamped config backup, per-tool idempotency guards, and a hardened post-install verification checklist.

Table of Contents

1. What This Is
 2. The Three Tools
 3. Architecture
 4. Install Guide
 5. Idempotency & Re-runs
 6. Post-Install Verification Reference
 7. Configuration File Reference
 8. Design Decisions
 9. Known Limitations
 10. Tested Versions
-

1. What This Is

`stack` is a Claude Code skill that orchestrates the installation of three complementary tools:

- **Longhand** — persistent local memory for Claude Code sessions
- **Context-Mode** — context window protection via tool output sandboxing
- **Hardgate** — enforcement layer blocking forbidden tool calls

Each tool has its own installer. `stack` handles the coordination: prerequisites, timestamped config backup, install order, idempotency checks, session state persistence across an interactive pause, and post-install verification.

Requirements

Requirement	Minimum version
Claude Code	Any
Python	3.10+
Node.js	18+
Shell	bash-compatible (Git Bash or WSL on Windows)

2. The Three Tools

Longhand — Memory Layer

Longhand stores every Claude Code session verbatim in a local SQLite + ChromaDB database. It exposes an MCP server that Claude can query for semantic recall across past sessions — finding relevant decisions, code changes, and context from previous conversations at ~126ms per query with zero API cost.

Install method: `pip install longhand`

Hooks registered:

- `SessionEnd` → `longhand ingest-session` — saves the session to the database on close
- `UserPromptSubmit` → `__prompt-hook-run` — records each prompt as it arrives

MCP server: `longhand mcp-server` — enables semantic search over stored sessions

Storage: All data lives locally in `~/ .longhand/` or equivalent. Nothing is transmitted externally.

Context-Mode — Context Layer

Context-Mode intercepts tool calls before their output reaches Claude's context window. Each intercepted call runs in a sandboxed subprocess; Claude receives a structured summary rather than the raw output dump. This prevents large command outputs (file reads, test runs, web fetches) from consuming the context window and forcing Claude to lose earlier conversation history.

Install method: `node install.js` from the Context-Mode repo directory

Hooks registered:

- `PreToolUse` with matcher `Bash` — sandboxes shell command output
- `PreToolUse` with matcher `Read` — sandboxes file read output
- `PreToolUse` with matcher `WebFetch` — sandboxes web fetch output
- (+ 6 additional matchers for git, npm, pytest, pip, Grep, Agent)

MCP server: `context-mode` — provides the sandbox execution environment

Required matchers for a valid install: `Bash`, `Read`, `WebFetch` must all be present. A partial install missing any of these is treated as a failure, not a warning.

Hardgate — Enforcement Layer

Hardgate installs a `PreToolUse` hook that exits with code 2 when Claude attempts to use a tool on the blocked list. Claude Code treats exit 2 as a hard stop — the tool call is cancelled and Claude reports the block to the user. Hardgate also installs a `SessionStart` hook that reminds Claude of the active enforcement rules at the start of each session.

Install method: `/hard-gate` skill (interactive — requires one decision from the user: which tool to enforce against)

Hooks registered:

- `PreToolUse` — exit 2 script (blocks forbidden tool calls)
- `SessionStart` — reminder script (informs Claude of active rules)

No blocked tools by default. Hardgate installs the enforcement infrastructure. The user configures which tools to block during the `/hard-gate` interactive step.

3. Architecture

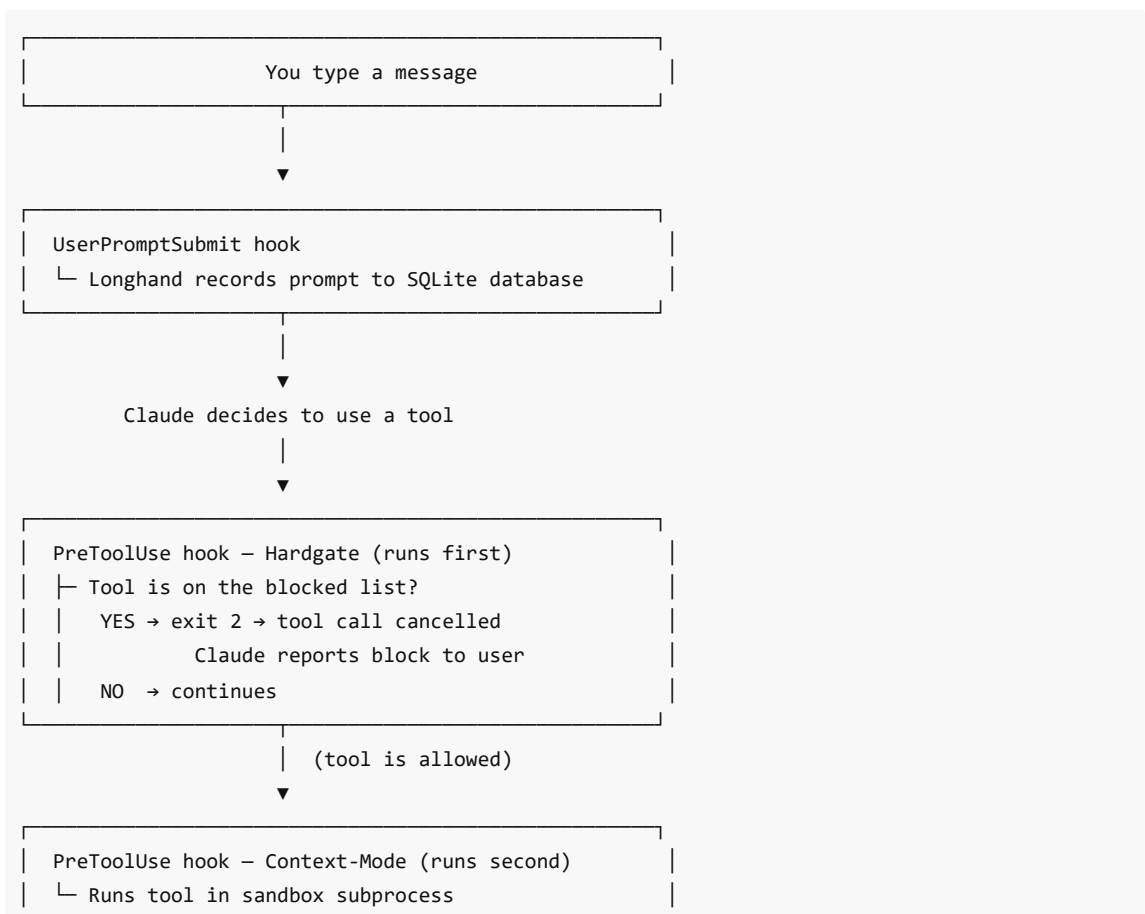
3.1 The Hook Event Model

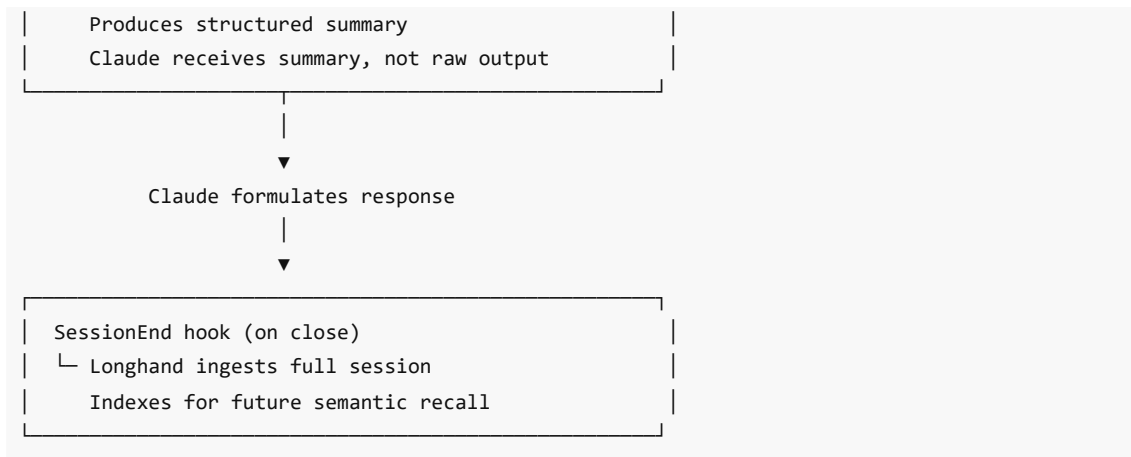
Claude Code fires hooks at specific lifecycle events. Each tool in the stack registers at a different event, so the three tools never conflict with each other.

EVENT	TOOL	ACTION
SessionStart	Hardgate	Reminds Claude of blocked tools
UserPromptSubmit (every message you send)	Longhand	Records prompt to local database
PreToolUse (before every tool call)	Hardgate Context-Mode	Checks tool against blocked list Runs tool in sandbox; returns summary instead of raw output
SessionEnd (on close)	Longhand	Ingests full session into searchable memory store

3.2 Request Flow

The following diagram shows the complete path of a single request from user input through tool execution to session persistence:





3.3 Configuration File Layout

All three tools write into `~/.claude/settings.json`. The stack installer creates a timestamped backup before any writes.

```
~/.claude/settings.json
├─ hooks
│   ├── SessionStart      ← Hardgate reminder script
│   ├── UserPromptSubmit ← Longhand prompt hook
│   ├── PreToolUse        ← Hardgate exit-2 script
│   │   ├── Context-Mode (Bash matcher)
│   │   ├── Context-Mode (Read matcher)
│   │   ├── Context-Mode (WebFetch matcher)
│   │   └── Context-Mode (+ 6 more matchers)
│   └── SessionEnd        ← Longhand ingest hook
└─ mcpServers
    ├── longhand          ← longhand mcp-server
    └── context-mode       ← context-mode sandbox server

~/.claude.json          ← alternate MCP server location
├─ mcpServers            ← checked as fallback if not in settings.json
│   ├── longhand
│   └── context-mode
```

3.4 Isolation Properties

The three layers are independent at runtime:

- **Hardgate** only reads its own block-list configuration. It does not know about Longhand or Context-Mode.
- **Context-Mode** only intercepts the tool calls that match its registered matchers. It does not know what Hardgate decided.
- **Longhand** only observes session events. It does not interact with tool execution at all.

If any one tool is misconfigured or removed, the other two continue to operate normally.

4. Install Guide

4.1 Install Phases

`/stack` runs four phases in sequence:

Phase 0 — Locate `verify.py`

Before doing anything else, the skill finds `scripts/verify.py` using three-stage discovery:

1. Check `$STACK_DIR/scripts/verify.py`
2. Search `~/.claude/plugins` for `*/stack/scripts/verify.py`
3. Bounded `find "$HOME" -maxdepth 4` as fallback

If `verify.py` cannot be found, the skill stops with a message telling the user to clone the full repo.

Phase 1 — Pre-flight

1. Check Python 3.10+ and Node.js 18+. Collect all failures before stopping — reports them all at once.
2. Create timestamped backups: `~/.claude/settings.json.stack-backup-YYYYMMDD-HHMMSS` and `~/.claude.json.stack-backup-YYYYMMDD-HHMMSS`
3. Run the idempotency check — determine which tools are fully installed and can be skipped.

Phase 2 — Install

Install each tool in order. Each is skipped only if ALL required artifacts are present.

1. Longhand: `pip install longhand → longhand setup → longhand prompt-hook install → claude mcp add longhand -s user -- longhand mcp-server`
2. Context-Mode: `node install.js` from the Context-Mode directory
3. Hardgate: interactive pause. Session variables are persisted to `~/.claude/stack-session.env` before the pause. User runs `/hard-gate` and selects Context-Mode as the enforcement target.

Phase 3 — Post-verify

After the user confirms Hardgate is done:

1. Source `~/.claude/stack-session.env` to restore session variables
2. Run `python "$VERIFY_PY"`
3. Interpret exit code and act (see Section 6)

4.2 Session State Persistence

The Hardgate install requires an interactive step that creates an interaction boundary — the skill pauses and waits for the user to run `/hard-gate` and reply. Variables set earlier in the session do not survive this boundary.

Before the pause, the skill writes:

```
cat > ~/.claude/stack-session.env <<EOF
VERIFY_PY=$VERIFY_PY
STACK_TS=$STACK_TS
CTX_DIR=$CTX_DIR
EOF
```

Phase 3 sources this file to restore `VERIFY_PY` and `STACK_TS` before running verification and the backup restore step.

5. Idempotency & Re-runs

`/stack` is safe to re-run at any time. Before installing any tool, it checks whether that tool is already fully installed.

Definition of "fully installed"

Longhand — all four must be true:

- `longhand` binary is on PATH
- `SessionEnd` hooks contain `longhand ingest-session`
- `UserPromptSubmit` hooks contain `__prompt-hook-run`
- `mcpServers` contains `longhand` (in either config file)

Context-Mode — all four must be true:

- `PreToolUse` hooks contain at least 3 Context-Mode matchers
- The required matchers `Bash` , `Read` , and `WebFetch` are all present
- No duplicate matchers
- `mcpServers` contains `context-mode` (in either config file)

If any artifact is missing, the full installer for that tool runs. There is no "partial skip" — a tool is either fully installed or it runs from scratch.

Each re-run creates a new timestamped backup, so multiple runs produce multiple independent snapshots.

6. Post-Install Verification Reference

`scripts/verify.py` is the post-install verifier. It can also be run standalone at any time:

```
python scripts/verify.py
```

Exit codes

Code	Meaning	Action
0	All required checks pass (Hardgate warning is acceptable)	Restart Claude Code
1	One or more required checks failed; config files are valid	Follow the retry commands printed
2	<code>settings.json</code> OR <code>.claude.json</code> exists but is malformed	Both files restored from timestamped backup

Checks performed

Check	Required	What it verifies
Longhand SessionEnd hook	Yes	<code>longhand ingest-session</code> present in SessionEnd hooks
Longhand UserPromptSubmit hook	Yes	<code>__prompt-hook-run</code> present in UserPromptSubmit hooks
Context-Mode PreToolUse hooks	Yes	≥ 3 matchers, <code>{Bash, Read, WebFetch}</code> $\subseteq$ present matchers, no duplicates
Longhand MCP server	Yes	<code>longhand</code> in <code>mcpServers</code> (either config file)
Context-Mode MCP server	Yes	<code>context-mode</code> in <code>mcpServers</code> (either config file)
Hardgate enforcement	Warning only	<code>.sh</code> file with "hardgate" content wired in hooks

Restore logic (exit 2)

When exit 2 is triggered, the skill restores both config files:

```
cp ~/.claude/settings.json.stack-backup-$STACK_TS ~/.claude/settings.json
cp ~/.claude.json.stack-backup-$STACK_TS ~/.claude.json
```

The restore hint in the verifier output uses `ls -t ... | head -1` to identify the most recent backup deterministically, even when multiple backups exist.

7. Configuration File Reference

Hook entry formats

Claude Code supports two hook entry formats. The verifier handles both.

Flat format:

```
{
  "command": "longhand ingest-session"
}
```

Wrapped format (nested hooks array):

```
{
  "hooks": [
    { "command": "longhand ingest-session" }
  ]
}
```

The `_command_str()` function in `verify.py` extracts the command text from either format for matching.

Context-Mode PreToolUse entry format

```
{
  "matcher": "Bash",
  "hooks": [
    { "command": "/path/to/context-mode/hook.sh" }
  ]
}
```

The `matcher` field is what the verifier checks. An entry without a `matcher` field does not count toward the required matcher set.

Backup naming

```
~/.claude/settings.json.stack-backup-20260416-143022
~/.claude.json.stack-backup-20260416-143022
```

The timestamp is `YYYYMMDD-HHMMSS` generated at the start of Phase 1 and shared by both backups from the same run. Multiple runs produce separate timestamped pairs. Backups are never overwritten.

8. Design Decisions

Why a skill, not a shell script?

A shell script would require the user to know where to find it and how to run it, and it can't coordinate with the interactive `/hard-gate` step which lives inside Claude Code. A Claude Code skill runs inside the session where the tools are needed, can prompt the user, and can pause and resume across interaction boundaries.

Why thin orchestration?

Each tool (Longhand, Context-Mode, Hardgate) owns its own install logic. The stack skill is deliberately thin — it handles coordination (order, backups, idempotency, verification) and defers everything else to the tool's own installer. This means fixes and updates to each tool's installer are automatically inherited by the stack without needing changes to the skill.

Why per-tool idempotency, not all-or-nothing?

A partial install is the most common failure mode — one command in a four-step sequence fails, the user fixes it and re-runs. An all-or-nothing check would force reinstalling tools that already succeeded. Per-tool idempotency means "skip only if ALL artifacts for this tool are present" — so a partial install triggers a full re-run of that tool's installer, which is the correct behavior.

Why symmetric rollback on exit 2?

`settings.json` and `.claude.json` are both Claude config files that MCP server registrations can land in depending on the tool. If either is malformed, Claude Code may fail to start or lose all its config. Restoring both files together (using the same timestamped backup pair) guarantees a consistent pre-install state regardless of which file was corrupted.

Why session state written to a file before the Hardgate pause?

Claude Code session variables don't survive the interaction boundary created when the skill pauses and waits for the user to complete an external action (`/hard-gate`). Writing `VERIFY_PY` , `STACK_TS` , and `CTX_DIR` to `~/.claude/stack-session.env` before the pause lets Phase 3 restore them reliably. The fallback (asking the user for the paths) is there if the file write fails, but in practice the file approach works on all tested platforms.

Why `find "$HOME" -maxdepth 4` as the fallback discovery?

The bounded depth prevents the search from running indefinitely on large home directories while still covering the common case of `~/stack/` , `~/projects/stack/` , `~/code/stack/` , etc. The search is filtered to `*/stack/scripts/verify.py` so it won't match an unrelated file named `verify.py` .

9. Known Limitations

Config presence ≠ MCP liveness

The verifier confirms that `longhand` and `context-mode` appear in `mcpServers` in the config. It does not confirm that those MCP server processes are running and responding. After install, verify runtime health separately:

```
longhand doctor
claude mcp list
```

Hardgate check is a warning, not a failure

The verifier emits a warning (⚠) rather than a failure (X) for Hardgate because Hardgate has no blocked tools until the user configures it. An install where Hardgate's infrastructure is in place but no target is set is a legitimate state.

Windows codepage

The verifier uses Unicode box-drawing and check-mark characters. On Windows terminals with narrow codepages (cp1252), these require `sys.stdout.reconfigure(encoding="utf-8")`, which is applied at startup. If output is still garbled, set `PYTHONUTF8=1` before running.

Tested version matrix

Behavior with versions outside the tested matrix is untested. See Section 10.

10. Tested Versions

This release was validated against the following versions:

Component	Tested version
stack	1.0.0
Longhand	0.5.5
Context-Mode	1.6.0
Python	3.14.3
Node.js	18+
OS	Windows 11 (Git Bash), macOS (Terminal)

Pinning Longhand:

```
pip install longhand==0.5.5
```

Pinning Context-Mode: Check out the validated commit hash from the `scottconverse/context-mode` repo before running `node install.js`.